

Diddar LMS

A reusable, multi-tenant learning platform starting with Diddar Bootcamp Cohort One.

```
Diddar
-> Diddar Bootcamp
    -> Software Engineering Program
        -> Cohort One
            -> Tracks -> Stages -> Lessons
```

This document covers backend architecture and implementation only. The first release is intentionally bootcamp-focused, but the model is designed to support universities, secondary schools, corporate training, and self-paced course platforms later.

Version 1.0 | August 2026

Status: Proposed implementation blueprint

1. Executive Summary

The product should be built as a hosted LMS platform. Diddar is the first customer organisation, not a hardcoded special case. A platform administrator provisions a private Diddar environment, applies a bootcamp template, and gives Diddar administrators the tools to create programs, cohorts, tracks, stages, lessons, applications, enrolments, and learning activity.

The central rule is: the platform owns capabilities, while each tenant configures how those capabilities are used.

The first release

- Create and configure a tenant for Diddar.
- Create a default Diddar Bootcamp learning environment.
- Create the Software Engineering Program.
- Create Cohort One with dates, capacity, instructors, and tracks.
- Allow learners to apply, be approved, and enrol.
- Deliver lessons, assignments, mentor sessions, progress tracking, notifications, and certificates.
- Give Diddar a private branded URL and admin console.

What is deliberately postponed

- Full university student information system replacement.
- Marketplace revenue sharing and creator payouts.
- Advanced AI tutoring and automated grading.
- Dedicated database infrastructure for enterprise tenants.
- Every possible school policy and integration.

2. Product Model

The system needs a small number of stable concepts. These concepts should be understandable to customers but also flexible enough to support different education models.

Concept	Meaning	Diddar example
Platform	The SaaS product operated by the LMS owner.	The reusable LMS platform.
Tenant	A customer organisation with isolated data, branding, users, and billing.	Diddar.
Learning environment	The customer-facing area where learning is organised. Internally this can be a workspace.	Diddar Bootcamp.
Program	A reusable learning path or curriculum.	Software Engineering Bootcamp.

Concept	Meaning	Diddar example
Offering	A scheduled or access-controlled instance of a program or course.	Cohort One.
Track	A path or specialisation inside an offering or program.	Frontend Engineering.
Stage	A sequence or milestone inside a track.	Web Foundations.
Course	Reusable learning content that may be used in multiple offerings.	HTML and CSS Fundamentals.
Lesson	A learner-facing unit of content or activity.	Semantic HTML.
Enrolment	The relationship between a learner and an offering.	Ada enrolled in Frontend, Cohort One.

Recommended customer-facing hierarchy

```

Diddar
  -> Bootcamp
    -> Program
      -> Cohort One
        -> Track
          -> Stage
            -> Lesson

```

The backend can use more neutral names such as learning environment and offering. The Diddar interface can display simpler labels such as Bootcamp and Cohort.

3. Multi-Tenancy Architecture

A tenant is the organisation that buys or operates an LMS instance. Each tenant receives isolated data and configurable presentation, but all tenants use the same application and core database model.

Tenant responsibilities

- Organisation identity and branding.
- Custom domain or subdomain.
- Tenant administrators, instructors, learners, and guardians where relevant.
- Programs, courses, cohorts, offerings, and learning records.
- Feature configuration and terminology.
- Reports, audit history, billing relationship, and integrations.

Tenant isolation

Request

- > Authenticate user
- > Resolve tenant from domain or selected context
- > Resolve learning environment
- > Check permission
- > Execute tenant-scoped query
- > Return only authorised data

Every tenant-owned table should carry `tenant_id`. Records that are scoped to a learning environment should also carry `environment_id`. The application must never accept a tenant identifier from the client as proof of access; it must derive tenant context from the authenticated session and verified domain or membership.

Database strategy

Option	Use	Recommendation
Shared database and shared schema	Every tenant-owned record has <code>tenant_id</code> .	Use for the initial release.
Shared database with row-level security	Database policies add a second isolation boundary.	Add when the data layer is stable.
Dedicated database per tenant	Separate infrastructure for enterprise customers.	Add later for regulated or very large customers.

Start with one database and strict tenant scoping. Design repository methods so moving a large tenant to a dedicated database later is possible without changing the product model.

4. Backend System Architecture

Use a modular monolith first. This keeps the product easier to develop, test, and deploy while preserving clear boundaries between business capabilities.

Web applications

- Public tenant sites
- Student application
- Instructor workspace
- Tenant admin console
- Platform admin console

Application backend

- Authentication and tenant context
- Tenant provisioning
- Catalog and curriculum
- Offerings and cohorts
- Applications and enrolments
- Learning activity
- Assessments
- Notifications
- Billing and reporting

Infrastructure

- PostgreSQL
- Object storage
- Background job queue
- Email and SMS provider
- Payment provider
- Observability and audit logs

Backend modules

Module	Primary responsibility
Identity	Users, authentication, sessions, verification, password reset, SSO later.
Tenants	Tenant creation, settings, branding, domains, feature flags, provisioning.
Membership	User membership, roles, permission assignments, scope checks.
Catalog	Programs, courses, tracks, stages, lessons, content versions.
Offerings	Cohorts, academic periods, schedules, capacity, instructors, publishing.
Admissions	Applications, approvals, waitlists, eligibility, status changes.
Enrolment	Access to offerings, transfers, withdrawals, completion state.
Learning	Progress, lesson access, assignments, projects, resources, attendance.
Assessment	Quizzes, submissions, grades, rubrics, feedback, certificates.
Communication	Announcements, notifications, templates, preferences, delivery.
Billing	Tenant plans, payments, invoices, discounts, refunds, payment state.
Reporting	Progress, attendance, outcomes, exports, dashboards, analytics.

Application layering

```
HTTP route / controller
-> Request validation
-> Tenant context resolver
-> Permission guard
-> Application service
-> Repository / transaction
-> Domain event
-> Response mapper
```

Route handlers should stay thin. Business rules such as whether a learner can enrol, whether a cohort is full, or whether a certificate can be issued belong in application services and domain rules.

5. Customer Sales and Provisioning

Diddar should be created through a controlled provisioning process. A sales lead must not automatically become a live tenant until the commercial and configuration work is complete.

```
Contact sales
-> sales_lead created
-> qualification and proposal
-> tenant created
-> bootcamp template applied
-> Diddar owner invited
-> branding and domain configured
-> program and cohort setup
-> tenant launched
```

Sales lead

- Organisation name and organisation type.
- Primary contact name, email, phone, and role.
- Estimated learner count and desired launch date.
- Required modules and integrations.
- Branding, custom domain, and content migration needs.
- Current LMS or student system.
- Notes, status, owner, and follow-up history.

Tenant provisioning record

```
tenant
  id: tenant_diddar
  name: Diddar
  type: bootcamp
  status: setup
  plan: managed

learning_environment
  id: env_diddar_bootcamp
  tenant_id: tenant_diddar
  name: Diddar Bootcamp
  status: setup
```

Provisioning checklist

Step	System action	Owner
1. Create tenant	Create isolated organisation record and default settings.	Platform admin
2. Apply template	Create roles, permissions, navigation, and bootcamp defaults.	System
3. Create environment	Create Diddar Bootcamp learning environment.	System
4. Invite owner	Send invitation to Diddar's first administrator.	System
5. Configure brand	Logo, colours, domain, email sender, certificate identity.	Diddar admin
6. Create catalog	Programs, tracks, stages, courses, lessons, and content.	Diddar admin
7. Create cohort	Dates, capacity, tracks, instructors, registration rules.	Diddar admin
8. Launch	Publish tenant and open registration.	Platform or tenant admin

6. Diddar Bootcamp Cohort One

This is the concrete first use case that the backend must support end to end.

```
Diddar
-> Diddar Bootcamp
  -> Software Engineering Program
    -> Cohort One
      -> Frontend Engineering
      -> Backend Engineering
      -> Product Design
```

Admin setup workflow

Order	Admin action	Created or changed
1	Create program	Software Engineering Program
2	Create tracks	Frontend, Backend, Product Design
3	Create levels	Beginner, Intermediate
4	Create stages	Foundations, Core Skills, Project Build
5	Add courses and lessons	Curriculum and learning content
6	Create Cohort One	Dates, capacity, status, registration rules
7	Attach tracks	The tracks available in Cohort One
8	Assign instructors	Staff responsible for delivery
9	Publish	Cohort becomes visible to learners

Learner workflow

```
Public Diddar page
-> View Cohort One
-> Create account
-> Choose track
-> Submit application
-> Application reviewed
-> Approved or waitlisted
-> Enrolment created
-> Welcome notification
-> Learner dashboard
-> Lessons, tasks, feedback, progress, certificate
```

Cohort state machine

```
draft
-> registration_open
-> registration_closed
-> active
-> completed
-> archived
```

State transitions must be controlled by service methods. For example, an archived cohort cannot accept applications, and a completed cohort cannot silently have its

grading rules changed.

Application state machine

```
started
-> submitted
-> under_review
-> approved
-> rejected
-> waitlisted
-> withdrawn
```

Enrolment rules

- A learner cannot have two active enrolments in the same track and cohort.
- A closed or archived cohort cannot receive a new enrolment.
- Capacity must be checked inside a transaction to prevent over-enrolment.
- Approval and payment requirements are tenant-configurable.
- A learner may be transferred between tracks only when policy allows it.
- Every enrolment status change should be auditable.

7. Data Model

The model below is the target domain for the first implementation. It is intentionally more complete than the first migration so future work does not require renaming core concepts.

Identity and tenancy tables

```
users
  id, email, name, avatar_url, status, created_at

tenants
  id, name, slug, type, status, plan, created_at

tenant_memberships
  id, tenant_id, user_id, status

learning_environments
  id, tenant_id, name, slug, status

environment_memberships
  id, environment_id, user_id, status

roles
  id, tenant_id, name, scope

permissions
  id, key

role_assignments
  id, role_id, user_id, tenant_id, environment_id
```

Catalog and curriculum tables

```
programs
  id, tenant_id, environment_id, title, description, status

tracks
  id, tenant_id, program_id, title, description, status

levels
  id, tenant_id, environment_id, title, rank

stages
  id, tenant_id, track_id, title, rank, completion_rule

courses
  id, tenant_id, title, description, status

course_versions
  id, course_id, version, status, published_at

modules
  id, course_version_id, title, rank

lessons
  id, module_id, title, type, rank, published_at

lesson_contents
  id, lesson_id, content_type, storage_key, body_json
```

Offering and enrolment tables

offerings

id, tenant_id, environment_id, program_id, title, type, status
starts_at, ends_at, registration_opens_at, registration_closes_at
capacity, approval_required, payment_required

offering_tracks

offering_id, track_id, level_id

offering_instructors

offering_id, user_id, role

applications

id, tenant_id, offering_id, user_id, track_id, status, submitted_at

enrolments

id, tenant_id, offering_id, user_id, track_id, status, enrolled_at

progress_records

id, tenant_id, enrolment_id, lesson_id, status, completed_at

Assessment and operations tables

```
assignments
submissions
grades
quizzes
quiz_questions
quiz_attempts
attendance_records
notifications
notification_deliveries
announcements
certificates
audit_logs
webhook_events
file_assets
```

All tenant-owned records need `tenant_id`. Environment-owned records should also carry `environment_id` where practical. Use foreign keys, unique constraints, and indexes to enforce the rules that matter most.

Constraint	Reason
Unique tenant slug	Stable tenant URL and lookup.
Unique tenant plus user membership	A user should not have duplicate memberships.
Unique offering plus user enrolment	Prevent duplicate enrolment.
Index <code>tenant_id</code> on every tenant table	Fast and safe tenant-scoped queries.
Index <code>offering_id</code> and <code>status</code>	Fast cohort dashboards and capacity checks.
Foreign keys with deliberate delete policy	Prevent accidental deletion of learning records.

8. API and Service Design

The API should be organised by business capability. The public site, admin console, student portal, and future mobile clients should use the same backend rules.

Initial API areas

```
/api/auth/*
/api/platform/*
/api/tenants/*
/api/environments/*
/api/memberships/*
/api/catalog/*
/api/programs/*
/api/courses/*
/api/offerings/*
/api/cohorts/*
/api/applications/*
/api/enrolments/*
/api/learning/*
/api/assessments/*
/api/notifications/*
/api/reports/*
```

Representative endpoints

Method and route	Purpose
POST /api/platform/tenants	Provision a tenant from a selected template.
GET /api/tenant	Return the current tenant context for an authenticated user.
POST /api/programs	Create a program inside the current environment.
POST /api/offerings	Create a cohort or other learning offering.
POST /api/offerings/:id/applications	Submit a learner application.
POST /api/applications/:id/approve	Approve and optionally enrol a learner.
GET /api/me/learning	Return the learner dashboard and active enrolments.
POST /api/lessons/:id/complete	Record learning progress.
POST /api/submissions	Submit an assignment or project.
GET /api/reports/cohorts/:id	Return cohort progress and operational metrics.

Service boundaries

```
TenantService
ProvisioningService
MembershipService
ProgramService
CourseService
OfferingService
ApplicationService
EnrolmentService
LearningProgressService
AssessmentService
NotificationService
CertificateService
ReportService
```

Services should expose actions, not raw database operations. For example, `approveApplication` should validate capacity, eligibility, permissions, payment state, and audit requirements before creating an enrolment.

9. Permissions, Security, and Compliance

The platform will contain learner identity data, grades, submissions, payment status, and organisation records. Security is a core feature of the backend, not a later enhancement.

Roles for the first release

Role	Scope and capabilities
Platform administrator	Provision tenants, support customers, manage platform settings.
Tenant owner	Manage Diddar settings, staff, branding, and tenant-level operations.

Role	Scope and capabilities
Environment administrator	Manage the Diddar Bootcamp catalog, cohorts, and learners.
Instructor	View assigned cohorts, deliver lessons, grade work, send announcements.
Reviewer	Review applications and grade assigned submissions.
Learner	View enrolled content, submit work, receive feedback, track progress.

Permission examples

```
tenant.manage_settings
environment.manage_members
program.create
course.publish
offering.create
offering.manage_enrolments
application.review
assessment.grade
report.view
certificate.issue
billing.manage
```

Required controls

- Resolve tenant context from session and domain, never from an untrusted request field.
- Check permission before every write and every sensitive read.
- Use signed URLs for private files and submitted work.
- Use rate limiting on authentication, applications, and public forms.
- Record audit events for provisioning, role changes, enrolment decisions, grading, exports, and deletion.
- Use soft deletion or archival for learning records instead of destructive deletion.
- Provide tenant data export and a documented tenant offboarding process.
- Back up the database and test restoration regularly.

10. Events, Notifications, and Background Jobs

Email, reminders, certificate generation, and reports should not block the main request. The backend should publish domain events and process non-critical work asynchronously.

```
application.submitted
application.approved
application.rejected
enrolment.created
cohort.published
lesson.completed
assignment.submitted
assignment.graded
cohort.started
cohort.completed
certificate.issued
```

Example event flow

```
ApplicationService.approve()
-> create enrolment in transaction
-> publish enrolment.created
-> notification job sends welcome email
-> dashboard job refreshes cohort metrics
-> audit job records approval
```

- Use an outbox table so database changes and emitted events cannot drift apart.
- Make jobs idempotent so retries do not send duplicate or conflicting results.
- Keep provider-specific email, SMS, and payment code behind integration interfaces.
- Store delivery status and provider message identifiers for support investigation.

11. Reusability for Other Customers

Diddar is the first tenant, but the platform should not assume every customer is a bootcamp. Reuse comes from configuration and extension points, not from making every workflow infinitely generic on day one.

Tenant configuration

- Branding: logo, colours, typography, domain, email identity, certificate identity.
- Terminology: cohort can display as intake, semester, term, or class group.
- Features: attendance, guardians, payments, certificates, discussions, live sessions.
- Workflows: auto-enrolment, approval, payment-before-access, application review.
- Custom fields: student ID, matriculation number, house, department, employee ID.
- Templates: bootcamp, university, secondary school, corporate training, marketplace.

Future mapping

Customer type	Tenant	Environment	Offering
Diddar bootcamp	Diddar	Diddar Bootcamp	Cohort One
University	University of Lagos	Computer Science	2026/2027 First Semester
Secondary school	Kings College	Senior School	2026 First Term

Customer type	Tenant	Environment	Offering
Marketplace	Platform or creator organisation	Public Catalog	Self-paced JavaScript course

The core offering model stays the same. What changes is the tenant template, terminology, permissions, fields, and workflow configuration.

12. Implementation Plan

Build in vertical slices. Each phase should produce a usable backend capability and should be tested with tenant isolation from the beginning.

Phase	Deliverables	Exit condition
0. Foundation	Database choice, migrations, configuration, error model, logging, test setup.	The application can safely connect to a development database.
1. Identity and tenancy	Users, login, tenants, environments, memberships, roles, tenant context.	A user can belong to Diddar and cannot access another tenant.
2. Provisioning	Sales leads, tenant creation, bootcamp template, invitations, branding settings.	A platform admin can provision Diddar without manual database work.
3. Catalog	Programs, tracks, levels, stages, courses, modules, lessons, publishing.	Diddar can build its curriculum through APIs/admin UI.
4. Cohorts and admissions	Offerings, cohort state, applications, capacity, approval, enrolment.	A learner can apply and become enrolled in Cohort One.
5. Learning engine	Progress, assignments, submissions, grading, attendance, notifications.	An instructor can run a real learning week.
6. Completion	Certificates, reports, exports, audit review, backups, support tools.	Diddar can complete a cohort and verify learner outcomes.
7. Expansion	University, secondary-school, marketplace templates and integrations.	A second customer type can be onboarded without changing the core model.

Recommended first engineering backlog

- Create the backend application and database package structure.
- Add migrations for users, tenants, environments, memberships, roles, and permissions.
- Implement authentication and tenant context middleware.
- Implement tenant-scoped repository helpers and tests.
- Implement platform tenant provisioning from a bootcamp template.
- Implement Diddar program, track, stage, course, lesson, and cohort APIs.
- Implement application approval and transactional enrolment.
- Add audit events and notification jobs before exposing public registration.

13. Suggested Backend Repository Structure

The current repository is a Next.js marketing site with small API routes. The backend can begin in the same repository, but its domain logic should be separated from the page components.

```
app/  
  api/  
    auth/  
    platform/  
    tenants/  
    environments/  
    programs/  
    offerings/  
    applications/  
    enrolments/  
    learning/  
  
src/  
  modules/  
    auth/  
    tenancy/  
    membership/  
    catalog/  
    offerings/  
    admissions/  
    enrolment/  
    learning/  
    assessment/  
    communication/  
    reporting/  
  db/  
    client.ts  
    schema/  
    migrations/  
  permissions/  
  events/  
  jobs/  
  integrations/  
  validation/  
  errors/  
  
tests/  
  unit/  
  integration/  
  security/  
  contract/
```

As the product grows, the API can move into a separate service without changing the domain model. The important boundary is between frontend rendering and backend application services.

14. Testing and Operations

Every feature that reads or writes tenant data needs isolation tests. Multi-tenancy is not complete until the system proves that a Diddar user cannot read University of Lagos data, even when identifiers are guessed.

Test layers

- Unit tests for domain rules and state transitions.
- Repository tests for constraints, indexes, and tenant scoping.

- Integration tests for application approval, capacity checks, and enrolment transactions.
- Security tests for cross-tenant access, role boundaries, signed URLs, and ID guessing.
- Contract tests for public APIs used by the student and admin applications.
- End-to-end tests for provisioning Diddar and completing a sample cohort journey.

Operational baseline

- Structured logs with request ID, tenant ID, user ID, and operation name.
- Error tracking with sensitive data redaction.
- Metrics for signups, applications, enrolments, job failures, and API latency.
- Database migration and rollback procedure.
- Daily backups and tested restore procedure.
- Health checks for database, object storage, email, and queue providers.
- Admin support tools for safe investigation and audited impersonation.

15. Decisions to Lock Before Coding

These decisions prevent expensive rewrites while keeping the first version focused.

Decision	Recommended choice
Tenant model	One tenant per customer organisation.
Environment model	One default learning environment per tenant, hidden as a technical detail initially.
First offering type	Cohort-based offering with dates and capacity.
Database	PostgreSQL with tenant_id on tenant-owned tables.
Architecture	Modular monolith before microservices.
Authentication	Use a trusted provider or well-tested auth package; do not build password security from scratch.
Files	Object storage with private signed access.
Async work	Outbox plus queue for notifications and reports.
Customisation	Tenant settings, templates, feature flags, terminology, and controlled custom fields.
First customer	Diddar Bootcamp Cohort One.

The backend should first make Diddar Cohort One operational. Reusability is achieved by making Diddar configurable and tenant-scoped, not by delaying delivery until every education model is perfect.

Appendix A. Complete Diddar Example

This is how the first provisioned tenant should look conceptually in the system.

Platform

Tenant: Diddar

type: bootcamp

status: active

domain: diddar.yourlms.com

Learning environment: Diddar Bootcamp

Program: Software Engineering Bootcamp

Tracks:

- Frontend Engineering
- Backend Engineering
- Product Design

Offering: Cohort One

start: 2027-01-12

end: 2027-04-05

capacity: 50

status: registration_open

Applications

- learner A: approved
- learner B: under_review

Enrolments

- learner A -> Frontend Engineering

Curriculum

Frontend Engineering

Stage: Web Foundations

Lesson: Semantic HTML

Lesson: CSS Layout

Assignment: Build a landing page

End of document.